

Design and optimize prompts

This exercise takes approximately **40 minutes**.

! **Note:** This lab assumes a pre-configured lab environment with Visual Studio Code, Azure CLI, and Python already installed.

Introduction

In this exercise, you'll test prompt optimizations for the Adventure Works Trail Guide Agent using a Git-based experimentation workflow. You'll first establish a quantified baseline using the current production prompt, then create experiment branches to test optimization variants. You'll run automated scripts to capture agent responses, manually score quality, and compare results to make evidence-based decisions about which optimization to deploy to production.

Scenario: You're operating the Adventure Works Trail Guide Agent with a v3 production prompt. Before optimizing, you'll measure baseline performance. Then you'll test if a token-optimized prompt (v4) can maintain quality while reducing costs by 40-50%. Finally, you'll test the same optimized prompt with the GPT-4.1-mini model to see if it can further reduce costs while maintaining acceptable quality.

Set up the environment

To complete the tasks in this exercise, you need:

- Visual Studio Code
- Azure subscription with Microsoft Foundry access
- Git and [GitHub](#) account
- Python 3.9 or later
- Azure CLI and Azure Developer CLI (azd) installed

! **Tip:** If you haven't installed these prerequisites yet, see [Lab 00: Prerequisites](#) for installation instructions and links.

All steps in this lab will be performed using Visual Studio Code and its integrated terminal.

Create repository from template

You'll start by creating your own repository from the template to practice realistic workflows.

1. In a web browser, navigate to the template repository on [GitHub](#) at `https://github.com/MicrosoftLearning/mslearn-genaiops`.
2. Select **Use this template** > **Create a new repository**.
3. Enter a name for your repository (e.g., `mslearn-genaiops`).
4. Set the repository to **Public** or **Private** based on your preference.
5. Select **Create repository**.

Clone the repository in Visual Studio Code

After creating your repository, clone it to your local machine.

1. In Visual Studio Code, open the Command Palette by pressing **Ctrl+Shift+P**.
2. Type **Git: Clone** and select it.
3. Enter your repository URL: `https://github.com/[your-username]/mslearn-genaiops.git`
4. Select a location on your local machine to clone the repository.
5. When prompted, select **Open** to open the cloned repository in VS Code.

Deploy Microsoft Foundry resources

Now you'll use the Azure Developer CLI to deploy all required Azure resources.

1. In Visual Studio Code, open a terminal by selecting **Terminal** > **New Terminal** from the menu.
2. Authenticate with Azure Developer CLI:

Code	Copy
<pre>azd auth login</pre>	

This opens a browser window for Azure authentication. Sign in with your Azure credentials.

3. Authenticate with Azure CLI:

Code	Copy
<pre>az login</pre>	

Sign in with your Azure credentials when prompted.

!

⚠ Important

In some environments, the VS Code integrated terminal may crash or close during the interactive login flow. If this happens, authenticate using explicit credentials instead:

Code	Copy
<pre>az login --username <your-username> --password <your-password></pre>	

4. Provision resources:

Code	Copy
<pre>azd up</pre>	

When prompted, provide:

- **Environment name** (e.g., `dev` , `test`) - Used to name all resources
- **Azure subscription** - Where resources will be created
- **Location** - Azure region (recommended: Sweden Central)

The command deploys the infrastructure from the `infra\` folder, creating:

- **Resource Group** - Container for all resources
- **Foundry (AI Services)** - The hub with access to models like GPT-4.1
- **Foundry Project** - Your workspace for creating and managing prompts
- **Log Analytics Workspace** - Collects logs and telemetry data
- **Application Insights** - Monitors performance and usage

5. Create a `.env` file with the environment variables:

Code	Copy
<pre>azd env get-values > .env</pre>	

!

⚠️

Important
– **File**
Encoding

After generating the `.env` file, make sure it is saved using **UTF-8** encoding.

In editors like **VS Code**, check the encoding indicator in the bottom-right corner.
If it shows **UTF-16 LE** (or any encoding other than UTF-8), click it, choose **Save with Encoding**, and select **UTF-8**.

Using the wrong encoding may cause environment variables to be read incorrectly.

This creates a `.env` file in your project root with all the provisioned resource information.

Install Python dependencies

With your Azure resources deployed, install the required Python packages.

1. In the VS Code terminal, create and activate a virtual environment:

CodeCopy

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

2. Install the required dependencies:

CodeCopy

```
python -m pip install -r requirements.txt
```

This installs necessary dependencies including:

- `azure-ai-projects` - SDK for working with Microsoft Foundry
- `azure-identity` - Azure authentication
- `python-dotenv` - Load environment variables

3. Add the agent configuration to your `.env` file:

Open the `.env` file in your repository root and add:

CodeCopy

```
AGENT_NAME="trail-guide"
MODEL_NAME="gpt-4.1"
```

Understand the experimental workflow

The repository contains the Trail Guide Agent, along with folders for testing and experiments:

CodeCopy

```
mslearn-genaiops/
├─ experiments/
│   ├─ baseline/
│   │   ├─ agent-responses.json      # Raw agent outputs
│   │   └─ evaluation.csv            # Manual quality scores
│   └─ optimized-concise/
│       ├─ agent-responses.json
│       └─ evaluation.csv
│   └─ gpt41mini/
│       ├─ agent-responses.json
│       └─ evaluation.csv
└─ src/
    ├─ agents/trail_guide_agent/
    │   ├─ trail_guide_agent.py      # Agent creation script (modify per branch)
    │   └─ prompts/
    │       ├─ v1_instructions.txt   # Basic prompt
    │       ├─ v2_instructions.txt   # Enhanced prompt
    │       ├─ v3_instructions.txt   # Production prompt (baseline)
    │       └─ v4_optimized_concise.txt # This lab: Token-optimized
    └─ tests/
        ├─ test-prompts/            # Test scenarios for evaluation
        │   ├─ day-hike-gear.txt     # Test: Essential day hike gear
        │   ├─ overnight-camping.txt # Test: First overnight camping
        │   ├─ three-day-backpacking.txt # Test: Extended trip safety
        │   ├─ winter-hiking.txt     # Test: Seasonal differences
        │   └─ trail-difficulty.txt  # Test: Beginner assessment
        └─ run_batch_tests.py        # Batch script to test agent with all prompts
```

Workflow per experiment:

1. Create experiment branch
2. Modify `trail_guide_agent.py` to point to the prompt variant being tested
3. Run `trail_guide_agent.py` to create/update agent version
4. Run `src/tests/run_batch_tests.py` to test agent with all prompts
5. Manually score responses in CSV
6. Compare results and merge winner

Establish baseline

Before testing optimizations, establish a baseline by deploying and evaluating the current production prompt (v3).

Why baseline first?

The baseline provides:

- **Reference point** - Quantify actual improvement from optimizations
- **Quality floor** - Ensure experiments don’t degrade below current performance
- **Token benchmark** - Measure cost reduction from optimization

Deploy baseline agent

1. Verify you’re on the main branch:

Code	Copy
git checkout main	

2. Open `src/agents/trail_guide_agent/trail_guide_agent.py` and make sure the baseline configuration uses the v3 prompt and GPT-4.1 model.

If you completed Lab 02 already, the file may already be configured for `v3_instructions.txt`. If you’re starting directly from the template repository, confirm these two settings before continuing:

The `prompt_file` assignment should be:

CodeCopy

```
prompt_file = Path(__file__).parent / 'prompts' / 'v3_instructions.txt'
```

The `model=` setting should be:

CodeCopy

```
model=os.getenv("MODEL_NAME", "gpt-4.1"),
```

3. If you changed the file in the previous step, save it and commit the baseline configuration on `main`:

!

⚠ First-time Git setup

If Git reports `Author identity unknown`, configure your identity once before committing:

CodeCopy

```
git config --global user.name "Your GitHub Username"
git config --global user.email "your-email@example.com"
```

CodeCopy

```
git add src/agents/trail_guide_agent/trail_guide_agent.py
git commit -m "Set baseline agent configuration to v3 prompt"
```

If the commit reports there is nothing to commit, the file already matched the required baseline configuration and you can continue.

4. Deploy the baseline agent:

CodeCopy

```
python src/agents/trail_guide_agent/trail_guide_agent.py
```

This creates the “trail-guide-agent” with your v3 prompt.

Run baseline tests

1. Run batch tests to capture baseline responses:

CodeCopy

```
python src/tests/run_batch_tests.py baseline
```

This tests all 5 prompts and saves results to `experiments/baseline/agent-responses.json`.

2. Review the captured responses:

CodeCopy

```
cat experiments/baseline/agent-responses.json
```

Score baseline responses

Create your baseline evaluation scores.

1. Check if it created or create `experiments/baseline/evaluation.csv` :

CodeCopy

```
test_prompt,agent_response_excerpt,intent_resolution,relevance,groundedness,comments
day-hike-gear,"For a summer day hike in moderate terrain, essential gear
includes...",5,5,4,Clear and comprehensive gear list
overnight-camping,"For your first overnight camping trip, start with these
essentials...",4,4,5,Good beginner advice with safety focus
three-day-backpacking,"Critical safety considerations for October mountain
backpacking...",5,5,5,Excellent detailed safety guidance
winter-hiking,"Winter hiking requires additional gear and precautions...",4,4,4,Good
comparison of seasonal differences
trail-difficulty,"To assess trail difficulty for your fitness level as a
beginner...",5,5,5,Helpful framework for self-assessment
```

Tip: Use the actual responses from `agent-responses.json` to score accurately.

You now have a quantified baseline to compare optimizations against.

Run optimization experiment 1: Token reduction

Test the optimized-concise prompt (v4) which reduces token usage while maintaining quality.

Create experiment branch

Each optimization experiment lives in its own branch, keeping experimental changes separate from your production agent.

1. In the VS Code terminal, ensure you’re in the repository root:

CodeCopy

```
cd /path/to/mslearn-genaiops
```

2. Ensure you’re on the main branch with latest changes:

CodeCopy

```
git checkout main
git pull origin main
```

3. Create an experiment branch for testing the optimized-concise prompt:

CodeCopy

```
git checkout -b experiment/optimized-concise
```

Note: Using `experiment/` prefix clearly identifies this as an experimental change.

Verify test prompts exist

The test prompts are already created in the `src/tests/test-prompts/` folder.

1. List the test prompt files:

CodeCopy

```
ls src/tests/test-prompts/
```

You should see:

- `day-hike-gear.txt` - Essential gear question
- `overnight-camping.txt` - First camping trip prep
- `three-day-backpacking.txt` - Extended trip safety
- `winter-hiking.txt` - Seasonal gear differences
- `trail-difficulty.txt` - Beginner trail assessment

2. Examine one of the test prompts:

CodeCopy

```
cat src/tests/test-prompts/day-hike-gear.txt
```

Output:

CodeCopy

```
What essential gear do I need for a summer day hike in moderate terrain?
```

These test prompts represent realistic user scenarios you'll use to evaluate each prompt variant.

Configure agent for the experiment

Modify the agent script to use the optimized-concise prompt (v4).

1. Open `src/agents/trail_guide_agent/trail_guide_agent.py` and update the `prompt_file` assignment to point to v4:

CodeCopy

```
# Change from:
prompt_file = Path(__file__).parent / 'prompts' / 'v3_instructions.txt'

# To:
prompt_file = Path(__file__).parent / 'prompts' / 'v4_optimized_concise.txt'
```

2. Save the file and commit this change to your experiment branch:

CodeCopy

```
git add src/agents/trail_guide_agent/trail_guide_agent.py
git commit -m "Configure agent to use v4 optimized-concise prompt"
```

If `git commit` reports there is nothing to commit, confirm the file already points to `v4_optimized_concise.txt` and then continue.

Deploy agent and run batch tests

Deploy the agent with the v4 prompt and capture responses from all test prompts.

1. Create/update the agent version from the repository root:

Code	Copy
<pre>python src/agents/trail_guide_agent/trail_guide_agent.py</pre>	

Expected output:

Code	Copy
<pre>Agent created (id: asst_abc123, name: trail-guide, version: 2)</pre>	

2. Run the batch test script to test with all prompts:

Code	Copy
<pre>python src/tests/run_batch_tests.py optimized-concise</pre>	

The script will:

- Find the deployed agent by name
- Run each test prompt through the agent
- Capture responses with token usage metadata
- Save results to `experiments/optimized-concise/agent-responses.json`

Expected output:

Code	Copy
------	------

```
Running 5 test prompts for experiment: optimized-concise

=====

Using agent: trail-guide (id: asst_abc123, version: 2)

Testing: day-hike-gear
  Prompt: What essential gear do I need for a summer day hike...
  Response captured (245 tokens)

Testing: overnight-camping
  Prompt: How should I prepare for my first overnight camping...
  Response captured (312 tokens)

Testing: three-day-backpacking
  Prompt: I'm planning a 3-day backpacking trip in the mountains...
  Response captured (385 tokens)

Testing: winter-hiking
  Prompt: What additional gear and precautions do I need for...
  Response captured (298 tokens)

Testing: trail-difficulty
  Prompt: How do I know if a trail is appropriate for my...
  Response captured (267 tokens)

=====

Results saved to: experiments/optimized-concise/agent-responses.json
Total tests: 5
Total tokens used: 1507
```

3. Review the captured responses:

Code	Copy
cat experiments/optimized-concise/agent-responses.json	

The JSON file contains structured data with test names, prompts, responses, and token usage.

Score responses manually

Review the agent responses and create an evaluation CSV with quality scores.

1. Create a new file `experiments/optimized-concise/evaluation.csv` :

Code	Copy
New-Item experiments/optimized-concise/evaluation.csv	

2. Open the file in VS Code and verify or add the CSV header and scores:

Code	Copy
------	------

```
test_prompt,agent_response_excerpt,intent_resolution,relevance,groundedness,comments
day-hike-gear,"Essential gear for summer moderate terrain day hikes...",5,5,4,Maintains
quality with reduced verbosity
overnight-camping,"First overnight camping essentials...",4,5,5,More direct while preserving
safety focus
three-day-backpacking,"October mountain backpacking safety priorities...",5,5,5,Excellent
concise safety guidance
winter-hiking,"Winter vs summer gear additions...",5,4,4,Clear comparison without excess
detail
trail-difficulty,"Beginner trail difficulty assessment...",5,5,5,Helpful and actionable
guidance
```

Scoring criteria (1-5 scale):

- **Intent Resolution:** Did the response address what the user asked?
- **Relevance:** Is the response on-topic and appropriate?
- **Groundedness:** Are claims factually accurate?

 **Tip:** Align your evaluation format with what Microsoft Foundry portal exports. Consistent criteria across manual testing, portal evaluations, and automated testing makes it easy to consolidate results.

3. Commit your experiment results:

CodeCopy

```
git add experiments/optimized-concise/
git commit -m "Complete optimized-concise experiment with evaluation"
git tag experiment-1-optimized-concise
```

Run optimization experiment 2: Model comparison

Test the same optimized prompt (v4) with the GPT-4.1-mini model to explore cost vs. quality tradeoffs.

Investigation goal

Determine if GPT-4.1-mini can maintain acceptable quality while providing additional cost savings beyond prompt optimization alone.

Run the experiment

1. Create a new experiment branch:

CodeCopy

```
git checkout experiment/optimized-concise
git checkout -b experiment/gpt41mini
```

2. Modify `src/agents/trail_guide_agent/trail_guide_agent.py` to use GPT-4.1-mini model.

Update the `model=` setting to change the model:

CodeCopy

```
# Change from:
model=os.getenv("MODEL_NAME", "gpt-4.1"),

# To:
model="gpt-4.1-mini",
```

Because this branch starts from `experiment/optimized-concise`, the `prompt_file` assignment should already point to `v4_optimized_concise.txt`. If it does not, update it before running the experiment.

3. Commit this configuration:

CodeCopy

```
git add src/agents/trail_guide_agent/trail_guide_agent.py
git commit -m "Configure agent to use GPT-4.1-mini model with v4 prompt"
```

4. Deploy the agent with GPT-4.1-mini:

CodeCopy

```
python src/agents/trail_guide_agent/trail_guide_agent.py
```

5. Run batch tests with the same test prompts:

CodeCopy

```
python src/tests/run_batch_tests.py gpt41mini
```

6. Create your evaluation CSV at `experiments/gpt41mini/evaluation.csv`:

CodeCopy

```
test_prompt,agent_response_excerpt,intent_resolution,relevance,groundedness,depth,comments
day-hike-gear,"For summer day hikes on moderate terrain...",4,4,4,3,Good coverage but less
detailed than GPT-4
overnight-camping,"First overnight camping essentials include...",4,4,4,3,Solid advice,
slightly less nuanced
three-day-backpacking,"October mountain safety priorities...",4,4,4,4,Good safety guidance,
concise
winter-hiking,"Winter hiking requires additional gear...",4,4,4,3,Clear comparison, adequate
detail
trail-difficulty,"To assess trail difficulty as a beginner...",4,4,4,4,Practical framework
provided
```

7. Commit experiment results:

CodeCopy

```
git add experiments/gpt41mini/
git commit -m "Complete GPT-4.1-mini experiment with evaluation"
git tag experiment-2-gpt41mini
```

Note: Keep the model change in `trail_guide_agent.py` committed on this branch. When you switch back to main, the script will revert to GPT-4.

Compare experiments and decide

After completing baseline and both optimization experiments, use your CSV data to make evidence-based decisions about which variant to promote to production.

Review all experiment branches

1. List all experiment branches:

CodeCopy

```
git branch | Select-String "experiment/"
```

You should see:

CodeCopy

```
experiment/gpt41mini
experiment/optimized-concise
```

2. Review the baseline results from `experiments/baseline/` on `main`, then compare them with the results captured on the two experiment branches.

Compare results across experiments

Use the CSV files to compare experiments side-by-side.

1. Open all three CSV files for comparison:

CodeCopy

```
code experiments/baseline/evaluation.csv
code experiments/optimized-concise/evaluation.csv
code experiments/gpt41mini/evaluation.csv
```

2. Review the scores across all three experiments to compare quality:

Look for patterns in the CSV data:

- Which experiment has consistently higher scores?
- Are there specific test prompts where quality differs significantly?
- How do token counts compare across experiments?

3. Make your decision based on the data:

Winner: `optimized-concise` **(v4 prompt with GPT-4)**

Rationale:

- Maintains or improves quality across test cases
- Significant token reduction (42% fewer tokens)
- Better cost-to-quality ratio than GPT-4.1-mini

Alternative consideration: GPT-4.1-mini could be used for simple, high-volume queries if cost is critical.

Merge winning experiment

Use Git to merge the winning experiment to main.

1. Checkout main branch:

CodeCopy

```
git checkout main
```

2. Merge the winning experiment:

```
``powershell git merge experiment/optimized-concise -m "Merge optimized-concise prompt experiment
```

Approved optimization reduces prompt tokens by 42% while maintaining quality.

- Validated with 5 production test cases
- Quality criteria met across all test prompts
- Cost savings: 42% reduction in completion tokens
- Ready for production deployment

Evaluation results: experiments/optimized-concise/evaluation.csv" ``

1. Verify the merged `trail_guide_agent.py` still uses the winning prompt:

Open `src/agents/trail_guide_agent/trail_guide_agent.py` and confirm the `prompt_file` assignment still points to `v4_optimized_concise.txt` :

CodeCopy

```
prompt_file = Path(__file__).parent / 'prompts' / 'v4_optimized_concise.txt'
```

If the merge completed without conflicts, this change should already be present and no additional edit is required.

2. If you had to resolve a merge conflict in the previous step, commit the resolved file:

CodeCopy

```
git add src/agents/trail_guide_agent/trail_guide_agent.py
git commit -m "Update agent to use optimized-concise prompt (v4)"
```

Otherwise, skip this commit because the merge commit already contains the prompt change.

3. Create a production release tag:

```
``powershell git tag -a v4-optimized-prompt -m "Release v4: Token-optimized prompts
```

Changes:

- Deployed v4_optimized_concise.txt prompt variant
- 42% token reduction validated
- Quality maintained across all test scenarios

Migration: Update trail_guide_agent.py to use v4_optimized_concise.txt" ``

1. Push changes to GitHub:

CodeCopy

```
git push origin main
git push origin experiment/optimized-concise
git push origin experiment/gpt41mini
git push --tags
```

Your experiments are now saved with full history on GitHub, making them easy to review and reference.

Next steps

- Continue to [Lab 04: Automated Evaluation](#) to scale your testing with automated evaluators

- Explore [Lab 05: Monitoring](#) to track production prompt performance
- Review [Lab 06: Tracing](#) to debug and optimize agent behavior